

Bootcamp: Engenheiro(a) de Dados

Desafio Final — Pipeline ETL Bronze / Silver / Gold

Integração Kafka Connect + PostgreSQL + Amazon S3 + Apache Spark

Aluno: Gabriel

Tema: Tesouro Direto (preços e taxas de títulos públicos)

1. Visão geral da solução

Este trabalho implementa um pipeline ETL completo no modelo Bronze, Silver e Gold, partindo dos dados abertos do Tesouro Direto e entregando dados prontos para análise em um data lake no Amazon S3. Toda a infraestrutura (PostgreSQL, plataforma Confluent/Kafka e Kafka Connect) foi provisionada com Docker, e o processamento das camadas analíticas foi feito com Apache Spark (Spark SQL API).

Fluxo de dados:

- Ingestão (Bronze): o CSV de preços e taxas do Tesouro Direto é carregado e tem suas colunas normalizadas, sendo gravado nas tabelas `dadostesouroipca` e `dadostesouropre` no PostgreSQL.
- Movimentação para o data lake: conectores Kafka Connect Source (JDBC) extraem os dados do PostgreSQL para tópicos no Kafka, e conectores Sink (S3) gravam esses dados em formato JSON, particionados, no bucket S3 (camada Bronze do data lake).
- Silver: com Apache Spark, os dados brutos do S3 são limpos — remoção de duplicações, conversão de timestamps para datas legíveis, tratamento de valores nulos — e gravados em Parquet.
- Gold: agregações por tipo de título (médias de preços unitários e contagem de registros) são geradas via consultas Spark SQL e gravadas em Parquet, prontas para consumo analítico.

Tecnologias utilizadas: Docker e docker-compose, PostgreSQL, Apache Kafka (plataforma Confluent), Kafka Connect (conectores JDBC e S3), Amazon S3, Apache Spark (PySpark / Spark SQL) e Jupyter Notebook.

2. Entregável 1 — Tabelas carregadas no PostgreSQL

As tabelas da camada Bronze foram criadas no PostgreSQL a partir do CSV do Tesouro Direto, já com os nomes de coluna normalizados (Tipo, Data_Vencimento, Data_Base, CompraManha, VendaManha, PUCompraManha, PUVendaManha, PUBaseManha, dt_update) e as datas convertidas para o tipo timestamp.

2.1. Estrutura da tabela dadotesouroipca

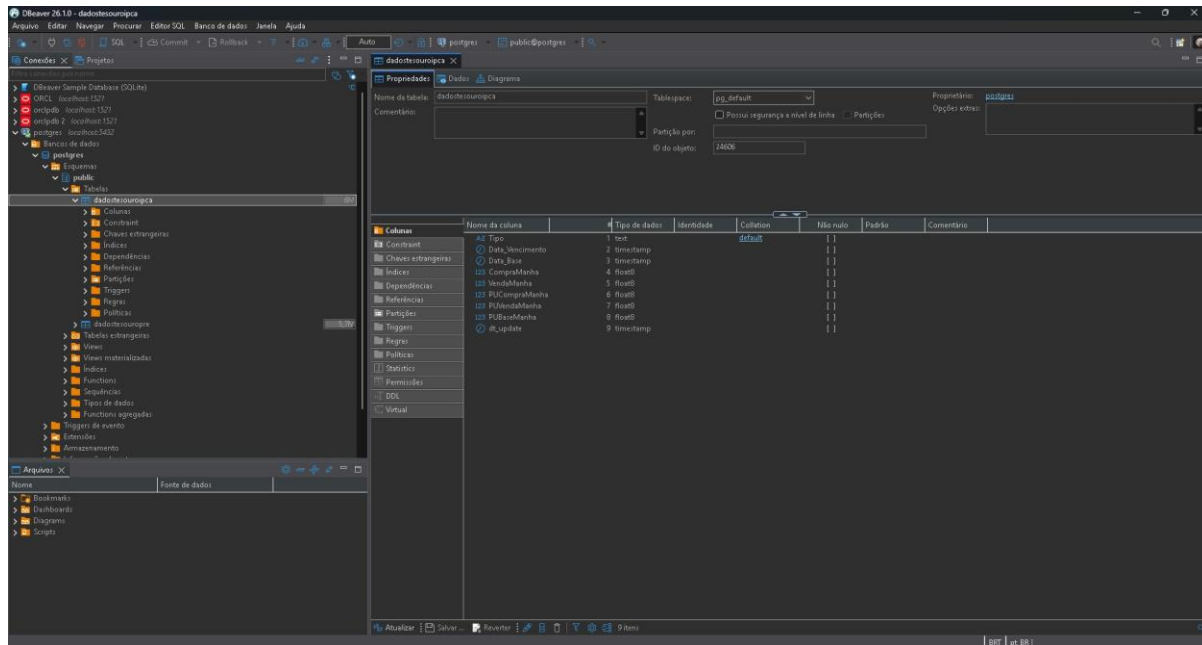


Figura 1 — Estrutura da tabela dadotesouroipca no DBeaver (colunas e tipos de dados).

2.2. Estrutura da tabela dadotesouropre

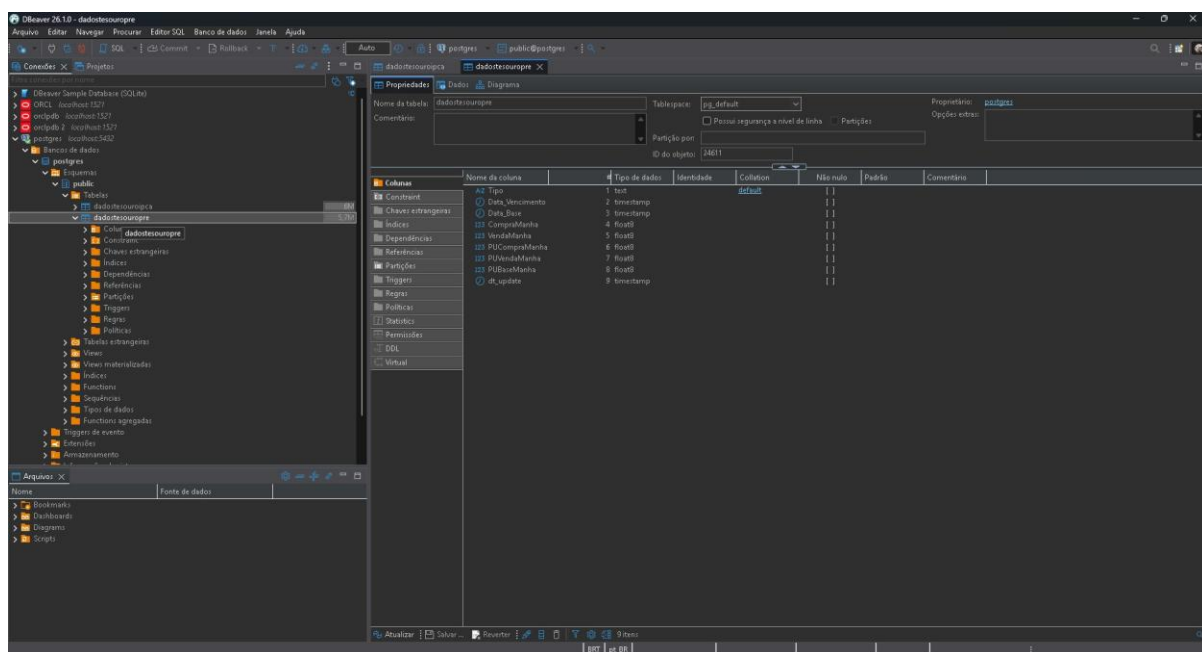


Figura 2 — Estrutura da tabela dadotesouropre no DBeaver.

2.3. Contagem de registros carregados

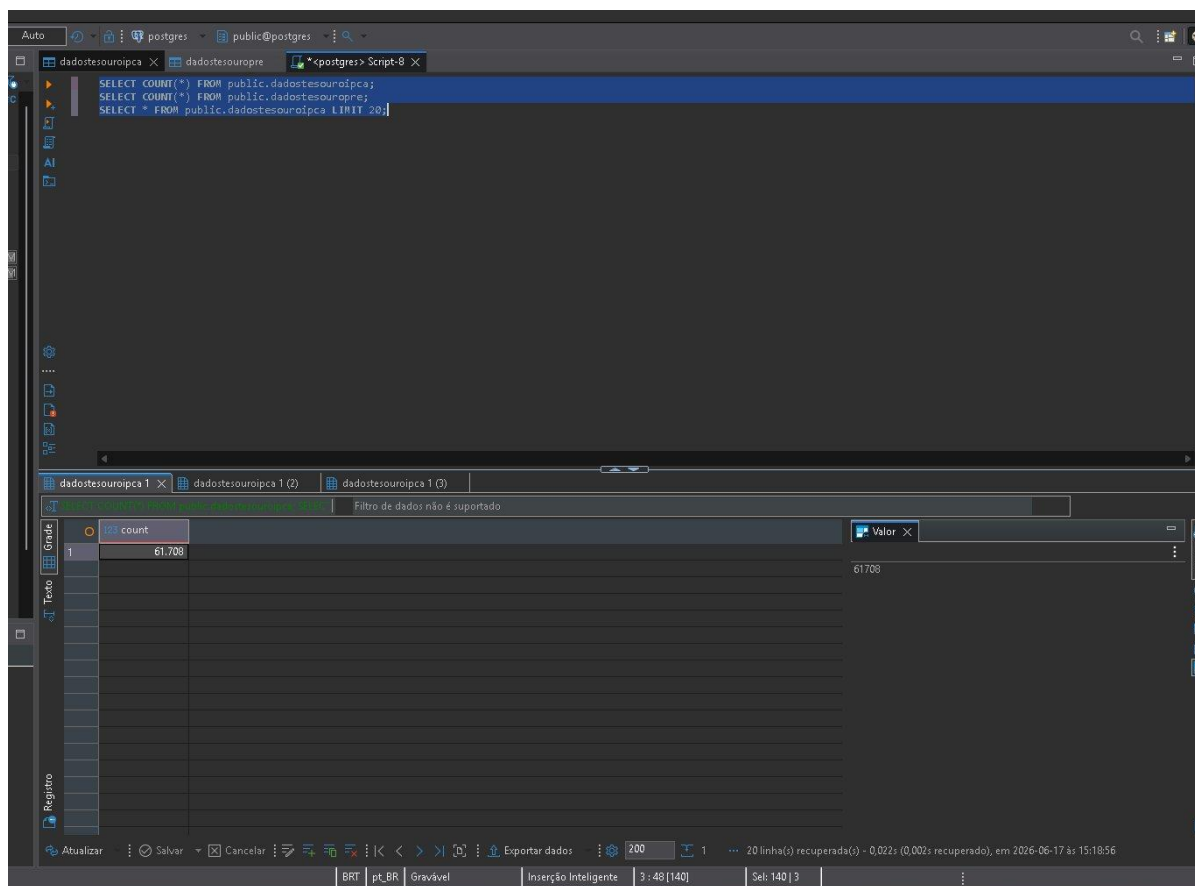


Figura 3 — Total de registros em dadotesouroipca.

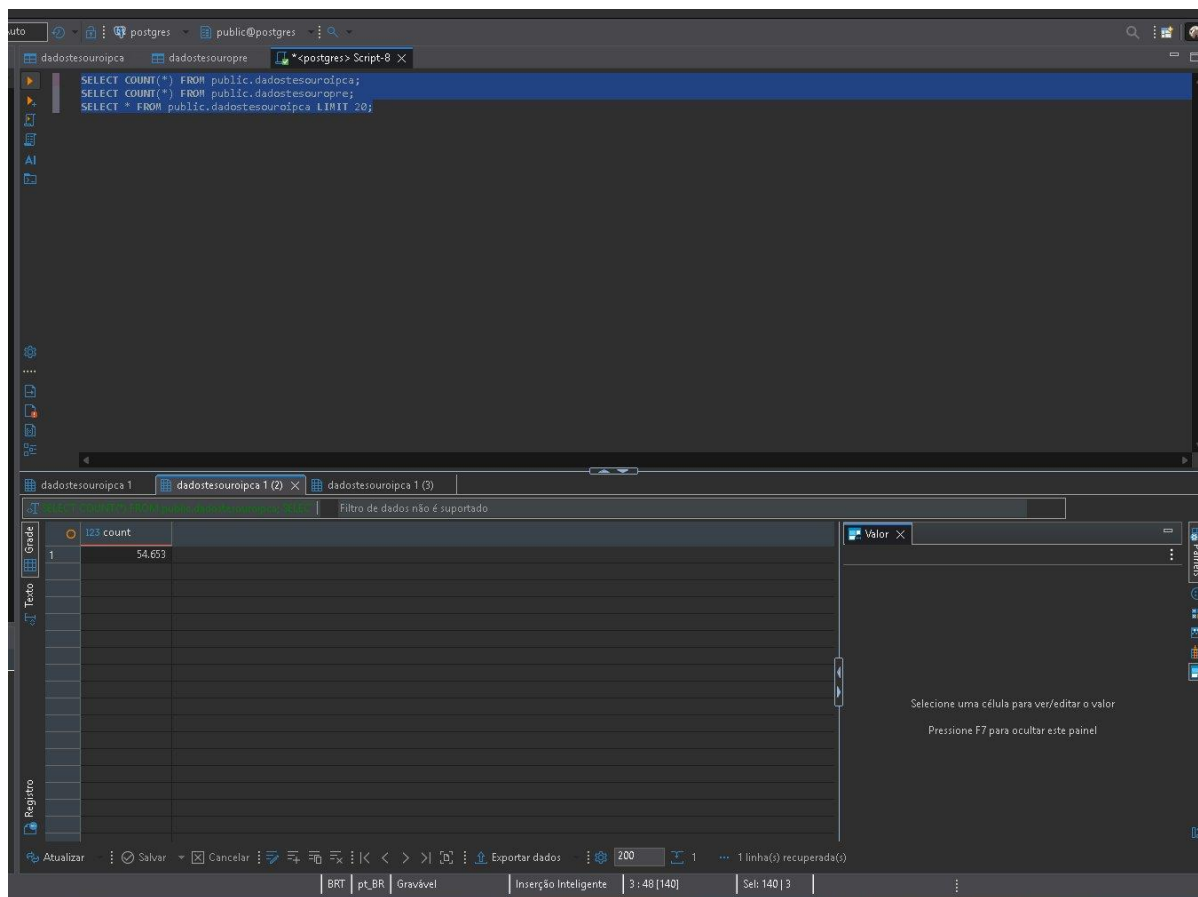


Figura 4 — Total de registros em dadostesouropre.

2.4. Amostra dos dados

SELECT COUNT(*) FROM public.dadostesouroipca;
SELECT COUNT(*) FROM public.dadostesouropre;
SELECT * FROM public.dadostesouroipca LIMIT 20;

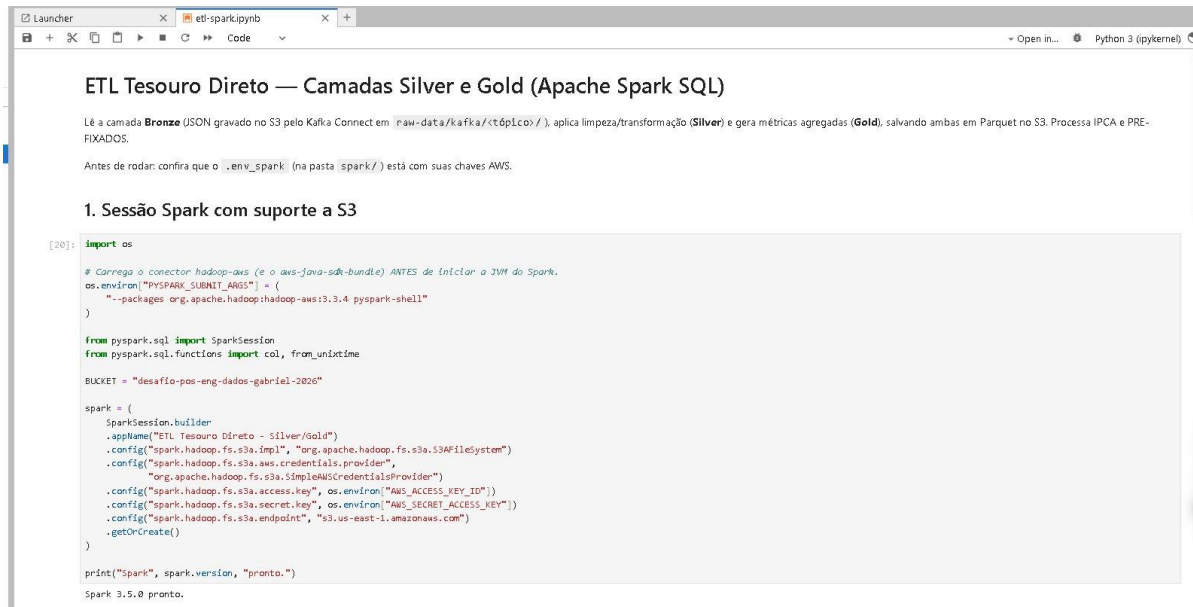
Id	Tipo	Data_Vencimento	Data_Base	123 ComprManha	123 VendaManha	123 PUComprManha	123 PUVendaManha	123 PUBaseManha	dt_atual
1	IPCA	2013-05-15 00:00:00.000	2011-05-18 00:00:00.000	6,58	6,6	2.025,71	2.024,98	2.024,15	2016-06-
2	IPCA	2045-05-15 00:00:00.000	2011-05-18 00:00:00.000	5,75	5,85	2.125,65	2.095,29	2.094,49	2016-06-
3	IPCA	2035-05-15 00:00:00.000	2011-05-18 00:00:00.000	5,86	5,96	2.086,37	2.060,65	2.059,85	2016-06-
4	IPCA	2024-08-15 00:00:00.000	2011-05-18 00:00:00.000	6,11	6,19	2.058,99	2.044,8	2.043,99	2016-06-
5	IPCA	2020-08-15 00:00:00.000	2011-05-18 00:00:00.000	6,34	6,4	2.010,66	2.022,55	2.021,73	2016-06-
6	IPCA	2017-05-15 00:00:00.000	2011-05-18 00:00:00.000	6,32	6,38	2.016,01	2.010,21	2.009,4	2016-06-
7	IPCA	2015-05-15 00:00:00.000	2011-05-18 00:00:00.000	6,53	6,57	2.009,82	2.007,11	2.006,29	2016-06-
8	IPCA	2012-08-15 00:00:00.000	2011-05-18 00:00:00.000	6,66	6,68	2.059,57	2.059,11	2.058,26	2016-06-
9	IPCA	2024-08-15 00:00:00.000	2011-05-18 00:00:00.000	6,02	6,1	945,18	935,81	935,45	2016-06-
10	IPCA	2015-05-15 00:00:00.000	2011-05-18 00:00:00.000	6,52	6,56	1.589,66	1.587,28	1.586,62	2016-06-
11	IPCA	2035-05-15 00:00:00.000	2011-05-18 00:00:00.000	5,61	5,71	554,41	541,01	541,8	2016-06-
12	IPCA	2024-08-15 00:00:00.000	2011-05-17 00:00:00.000	6,14	6,22	2.052,85	2.038,71	2.037,9	2016-06-
13	IPCA	2020-08-15 00:00:00.000	2011-05-17 00:00:00.000	6,37	6,43	2.025,78	2.017,69	2.016,87	2016-06-
14	IPCA	2045-05-15 00:00:00.000	2011-05-17 00:00:00.000	5,77	5,87	2.118,72	2.098,5	2.087,7	2016-06-
15	IPCA	2035-05-15 00:00:00.000	2011-05-17 00:00:00.000	5,89	5,99	2.077,8	2.052,22	2.051,43	2016-06-
16	IPCA	2012-08-15 00:00:00.000	2011-05-17 00:00:00.000	6,65	6,67	2.058,96	2.058,49	2.057,64	2016-06-
17	IPCA	2013-05-15 00:00:00.000	2011-05-17 00:00:00.000	6,56	6,58	2.025,6	2.024,87	2.024,06	2016-06-

Figura 5 — Amostra dos dados carregados (SELECT * com as colunas normalizadas).

3. Entregável 2 — Código Spark e consultas Spark SQL

O processamento das camadas Silver e Gold foi feito com Apache Spark, executado em um contêiner Docker com Jupyter Notebook. A leitura e a escrita ocorrem diretamente no Amazon S3 via conector s3a.

3.1. Sessão Spark com suporte a S3



```
ETL Tesouro Direto — Camadas Silver e Gold (Apache Spark SQL)

Lê a camada Bronze (JSON gravado no S3 pelo Kafka Connect em raw-data/kafka/<tópico>/), aplica limpeza/transformação (Silver) e gera métricas agregadas (Gold), salvando ambas em Parquet no S3. Processa IPCA e PRE-FIXADOS.

Antes de rodar, confira que o .env_spark (na pasta spark/) está com suas chaves AWS.

1. Sessão Spark com suporte a S3

[20]: import os

# Carrego o conector hadoop-aws (e o aws-java-sdk-bundle) ANTES de iniciar a JVM do Spark.
os.environ["PYSPARK_SUBMIT_ARGS"] = (
    "--packages org.apache.hadoop:hadoop-aws:3.3.4 pyspark-shell"
)

from pyspark.sql import SparkSession
from pyspark.sql.functions import col, from_unixtime

BUCKET = "desafio-pos-eng-dados-gabriel-2020"

spark = (
    SparkSession.builder
    .appName("ETL Tesouro Direto - Silver/Gold")
    .config("spark.hadoop.fs.s3a.impl", "org.apache.hadoop.fs.s3a.S3AFileSystem")
    .config("spark.hadoop.fs.s3a.aws.credentials.provider",
            "org.apache.hadoop.fs.s3a.SimpleAWSCredentialsProvider")
    .config("spark.hadoop.fs.s3a.access.key", os.environ["AWS_ACCESS_KEY_ID"])
    .config("spark.hadoop.fs.s3a.secret.key", os.environ["AWS_SECRET_ACCESS_KEY"])
    .config("spark.hadoop.fs.s3a.endpoint", "s3.us-east-1.amazonaws.com")
    .getOrCreate()

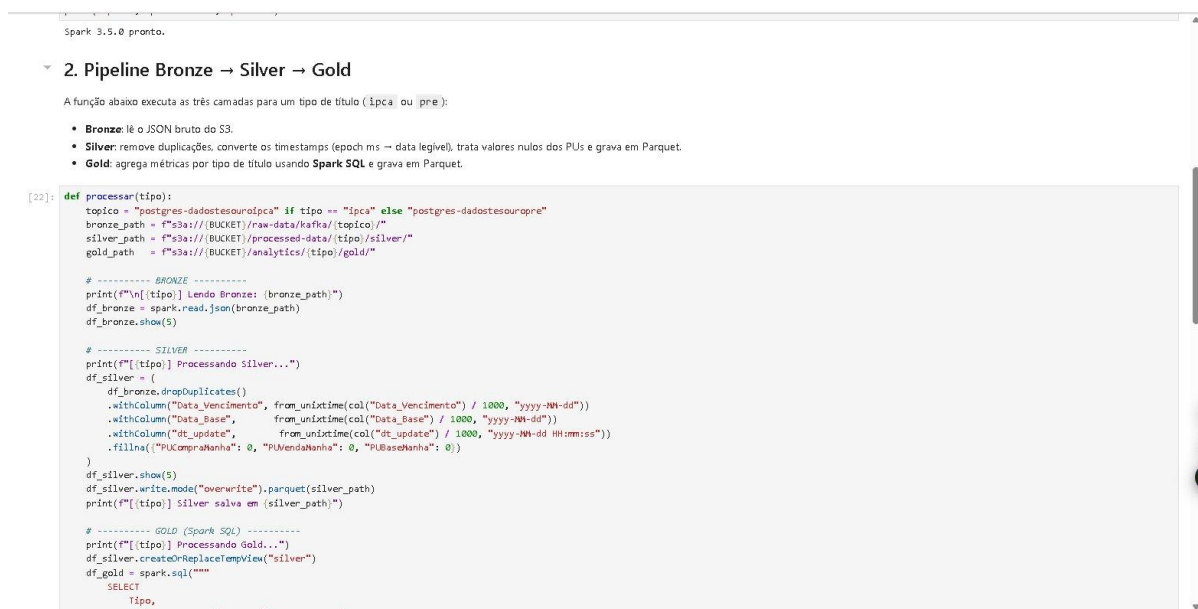
    print("Spark", spark.version, "pronto.")

Spark 3.5.0 pronto.
```

Figura 6 — Inicialização da SparkSession com o conector hadoop-aws e configuração de acesso ao S3 (Spark 3.5.0).

3.2. Pipeline Bronze → Silver → Gold (incluindo a consulta Spark SQL)

A camada Silver remove duplicações (dropDuplicates), converte os timestamps em datas legíveis e trata valores nulos. A camada Gold é gerada por uma consulta Spark SQL que agrega as métricas por tipo de título.



```
Spark 3.5.0 pronto.

2. Pipeline Bronze → Silver → Gold

A função abaixo executa as três camadas para um tipo de título (ipca ou pre):

• Bronze: lê o JSON bruto do S3.
• Silver: remove duplicações, converte os timestamps (epoch ms → data legível), trata valores nulos dos PUs e grava em Parquet.
• Gold: agrega métricas por tipo de título usando Spark SQL e grava em Parquet.

[22]: def processar(tipo):
    topico = "postgres-dadosesouroipca" if tipo == "ipca" else "postgres-dadosesourpre"
    bronze_path = f"s3a://{BUCKET}/raw-data/kafka/{topico}/"
    silver_path = f"s3a://{BUCKET}/processed-data/{tipo}/silver/"
    gold_path = f"s3a://{BUCKET}/analytics/{tipo}/gold/"

    # ----- BRONZE -----
    print(f"[{tipo}] Lendo Bronze: {bronze_path}")
    df_bronze = spark.read.json(bronze_path)
    df_bronze.show(5)

    # ----- SILVER -----
    print(f"[{tipo}] Processando Silver...")
    df_silver = (
        df_bronze.dropDuplicates()
        .withColumn("Data_Vencimento", from_unixtime(col("Data_Vencimento") / 1000, "yyyy-MM-dd"))
        .withColumn("Data_Base", from_unixtime(col("Data_Base") / 1000, "yyyy-MM-dd"))
        .withColumn("dt_update", from_unixtime(col("dt_update") / 1000, "yyyy-MM-dd HH:mm:ss"))
        .fillna({"PUCompraManha": 0, "PUVendaManha": 0, "PUBaseManha": 0})
    )
    df_silver.show(5)
    df_silver.write.mode("overwrite").parquet(silver_path)
    print(f"[{tipo}] Silver salva em {silver_path}")

    # ----- GOLD (Spark SQL) -----
    print(f"[{tipo}] Processando Gold...")
    df_silver.createOrReplaceTempView("silver")
    df_gold = spark.sql("""
        SELECT
            Tipo,
            ...
    """)
```

Figura 7 — Código das transformações Silver e da consulta Spark SQL (SELECT ... GROUP BY Tipo) da camada Gold.

3.3. Execução do pipeline (IPCA)

3. Executar para IPCA

```
[29]: df_silver_ipca, df_gold_ipca = processar("ipca")
```

[ipca] Lendo Bronze: s3a://desafio-pos-eng-dados-gabriel-2026/raw-data/kafka/postgres-dadosesouroipca/

CompraNanha	Data_Base	Data_Vencimento	PUBaseNanha	PUCompraNanha	PUVendaNanha	Tipo	VendaNanha	dt_update	partition
10.25	1153180800000	1218758400000	1515.63	1517.37	1516.32	IPCA	10.29	1781708649926	0
10.19	1153180800000	1242345600000	1456.59	1458.61	1457.24	IPCA	10.23	1781708649926	0
10.14	1153180800000	1305417600000	1372.46	1375.14	1373.08	IPCA	10.18	1781708649926	0
10.12	1152662400000	1305417600000	1371.43	1373.93	1371.86	IPCA	10.16	1781708649926	0
10.19	1152662400000	1281830400000	1421.32	1423.6	1421.77	IPCA	10.23	1781708649926	0

only showing top 5 rows

[ipca] Processando Silver...

CompraNanha	Data_Base	Data_Vencimento	PUBaseNanha	PUCompraNanha	PUVendaNanha	Tipo	VendaNanha	dt_update	partition
10.24	2006-07-11	2009-05-15	1452.2	1454.04	1452.66	IPCA	10.28	2026-06-17 15:04:09	0
8.82	2006-07-04	2045-05-15	1116.54	1129.15	1116.89	IPCA	8.92	2026-06-17 15:04:09	0
10.29	2006-07-17	2008-08-15	1513.9	1515.63	1514.58	IPCA	10.33	2026-06-17 15:04:09	0
10.88	2006-07-10	2006-08-15	1624.7	1625.28	1625.25	IPCA	10.9	2026-06-17 15:04:09	0
10.9	2006-06-29	2009-05-15	1427.0	1428.9	1427.54	IPCA	10.94	2026-06-17 15:04:09	0

only showing top 5 rows

[ipca] Silver salva em s3a://desafio-pos-eng-dados-gabriel-2026/processed-data/ipca/silver/
[ipca] Processando Gold...

Tipo	Media_PUCompraNanha	Media_PUVendaNanha	Media_PUBaseNanha	Total_Registros
IPCA	3028.880060566533	3000.7285911293347	3000.744830724316	32196

[ipca] Gold salva em s3a://desafio-pos-eng-dados-gabriel-2026/analytics/ipca/gold/

4. Executar para PRE-FIXADOS

Figura 8 — Execução para o IPCA: leitura da Bronze, dados da Silver (datas convertidas) e resultado da consulta Spark SQL da camada Gold.

3.4. Execução do pipeline (PRE-FIXADOS)

4. Executar para PRE-FIXADOS

```
[29]: df_silver_pre, df_gold_pre = processar("pre")
```

[pre] Lendo Bronze: s3a://desafio-pos-eng-dados-gabriel-2026/raw-data/kafka/postgres-dadosesouropre/

CompraNanha	Data_Base	Data_Vencimento	PUBaseNanha	PUCompraNanha	PUVendaNanha	Tipo	VendaNanha	dt_update	partition
10.71	1640736000000	1924992000000	1004.74	1011.49	1004.74	PRE-FIXADOS	10.83	1781708649926	0
10.61	1640736000000	1796761600000	1022.24	1026.95	1022.24	PRE-FIXADOS	10.73	1781708649926	0
10.66	1640736000000	1861920000000	1013.53	1019.38	1013.53	PRE-FIXADOS	10.78	1781708649926	0
10.63	1640649600000	1861920000000	1014.47	1020.34	1014.47	PRE-FIXADOS	10.75	1781708649926	0
10.65	1640649600000	1735689600000	1029.57	1032.83	1029.57	PRE-FIXADOS	10.77	1781708649926	0

only showing top 5 rows

[pre] Processando Silver...

CompraNanha	Data_Base	Data_Vencimento	PUBaseNanha	PUCompraNanha	PUVendaNanha	Tipo	VendaNanha	dt_update	partition
10.74	2021-12-09	2025-01-01	1021.99	1025.29	1021.99	PRE-FIXADOS	10.86	2026-06-17 15:04:09	0
10.52	2017-06-22	2025-01-01	1016.77	1022.93	1017.18	PRE-FIXADOS	10.64	2026-06-17 15:04:09	0
10.75	2023-06-27	2031-01-01	1006.47	1012.55	1006.47	PRE-FIXADOS	10.87	2026-06-17 15:04:09	0
10.57	2017-06-26	2025-01-01	1015.2	1021.34	1015.61	PRE-FIXADOS	10.69	2026-06-17 15:04:09	0
10.42	2021-12-23	2027-01-01	1027.42	1032.58	1027.42	PRE-FIXADOS	10.54	2026-06-17 15:04:09	0

only showing top 5 rows

[pre] Silver salva em s3a://desafio-pos-eng-dados-gabriel-2026/processed-data/pre/silver/
[pre] Processando Gold...

Tipo	Media_PUCompraNanha	Media_PUVendaNanha	Media_PUBaseNanha	Total_Registros
PRE-FIXADOS	886.2400398931143	885.757259506808	885.856329434971	39498

[pre] Gold salva em s3a://desafio-pos-eng-dados-gabriel-2026/analytics/pre/gold/

5. Conferência final (camada Gold)

Figura 9 — Execução para o PRE-FIXADOS: leitura da Bronze, dados da Silver e resultado da consulta Spark SQL da camada Gold.

4. Entregável 3 — Dados no Amazon S3 (organizados e particionados)

O bucket S3 organiza as três camadas do pipeline em prefixos distintos: raw-data (Bronze), processed-data (Silver) e analytics (Gold). Os dados brutos chegam particionados por partição do tópico Kafka (partition=0).

4.1. Organização do bucket nas três camadas

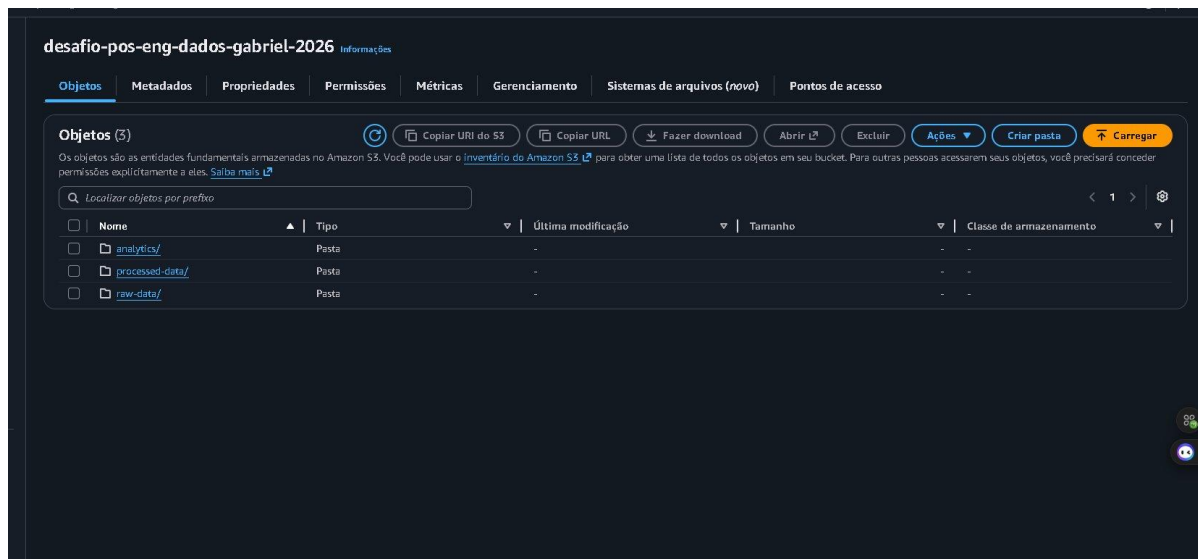


Figura 10 — Bucket S3 com as pastas analytics/ (Gold), processed-data/ (Silver) e raw-data/ (Bronze).

4.2. Camada Bronze — dados brutos particionados

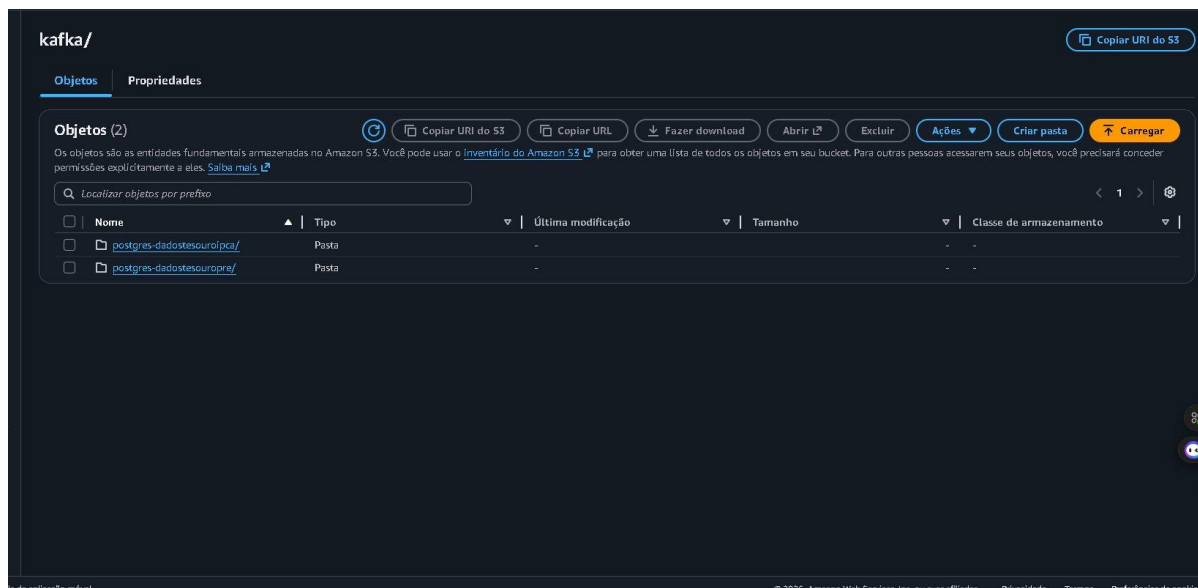


Figura 11 — Pasta raw-data/kafka/ com os dois tópicos (postgres-dadostesouroipca e postgres-dadostesouopre).

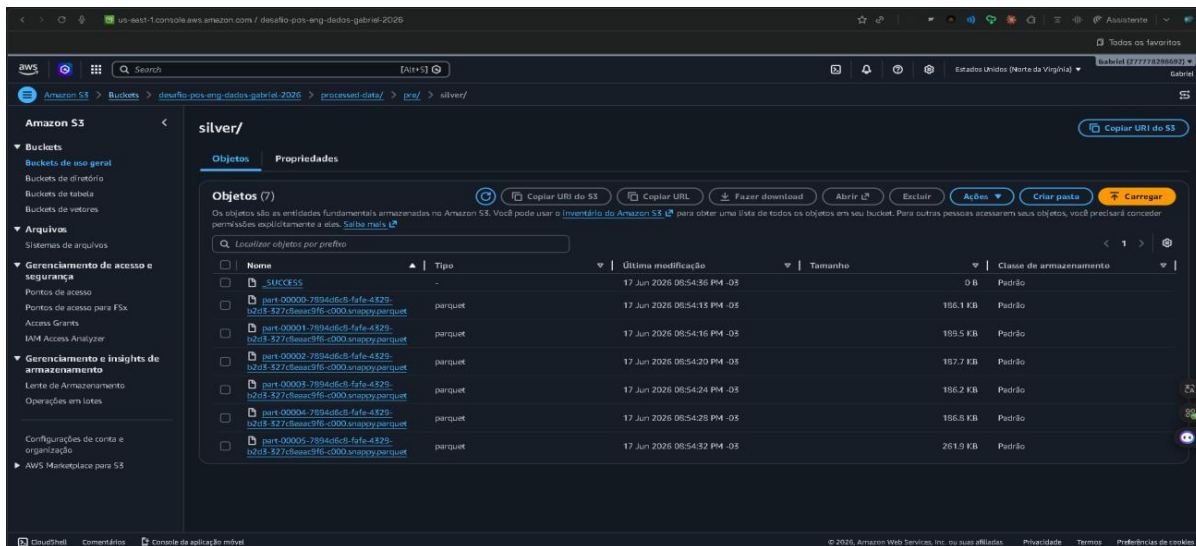


Figura 14 — Camada Silver do PRE-FIXADOS em Parquet (processed-data/pre/silver/).

4.4. Camada Gold (Parquet)

A camada Gold é gravada em Parquet no prefixo analytics/ (analytics/ipca/gold/ e analytics/pre/gold/). Sua persistência está comprovada pelos logs de execução do Spark (Figuras 8 e 9), que registram “Gold salva em s3a://.../analytics/.../gold/” ao final de cada pipeline, e pela própria estrutura do bucket (Figura 10), onde a pasta analytics/ aparece como uma das três camadas do data lake.

5. Anexo — Evidências do pipeline Kafka Connect

Como evidência complementar da movimentação dos dados do PostgreSQL para o S3, abaixo o status dos conectores Kafka registrados, todos em estado RUNNING.

```
@6a380000@-
[3/3] RUN confluent-hub install --no-prompt confluentinc/kafka-connect-jdbc:10.4.2
#7 CACHED
#8 exporting to image
#8 exporting layers 0.0s done
#8 exporting manifest sha256:2ead27b0d09093e5dbcc8cf412cc944c4d87950b9e7a8b3cfd70eff3a5 done
#8 exporting config sha256:6151a1f69519835cc08f4a9a99b10a49a2e0722020f10961a3c5f7a31672 done
#8 exporting attestation manifest sha256:9f6a26a92f940b1452f5db0bece1a52d1f5ad2c5b2f1465fdaa214de90315 0.0s done
#8 exporting manifest list sha256:432d20c79ed127a3a4e21d8452e38575f2508972320e20e74130d9c57acbd0 0.0s done
#8 naming to docker.io/library/connect-custom:7.6.1 0.0s done
#8 unpacking to docker.io/library/connect-custom:7.6.1 0.0s done
#8 DONE 0.1s
#9 resolving provenance for metadata file
#9 DONE 0.0s
[+] up 5/5
# Image connect-custom:7.6.1 built
# Container zookeeper Running 2/3s
# Container connect Started 0.0s
# Container broker Started 1/2s
# Container schema-registry Started 0.0s
# C:\Users\gabril\OneDrive\Documents\DOCS\VESTUO5\PO5 XP\ENGENHEIRO DE DADOS\DESAFIO\projeto_desafio_final\confluent curl.exe http://localhost:8083/connectors
# [3] curl_pre "jdbc.source.postgres.pre" "jdbc.source.postgres.ipca" "jdbc.ipca"
# C:\Users\gabril\OneDrive\Documents\DOCS\VESTUO5\PO5 XP\ENGENHEIRO DE DADOS\DESAFIO\projeto_desafio_final\confluent curl.exe -X POST -H "Content-Type: application/json" --data "@./connectors/source/connect_jdbc_postgres.ipca.config.js" http://localhost:8083/connectors
# curl.exe -X POST -H "Content-Type: application/json" --data "@./connectors/source/connect_jdbc_postgres.pre.config.js" http://localhost:8083/connectors
# {"name":"postg-conector-ipc","config":{"connector.class":"io.confluent.connect.jdbc.JdbcSourceConnector","tasks.max":"1","connection.url":"jdbc:postgresql://postgres:5432/postgres","connection.user":"postgres","connection.password":"postgres","mode":"timestamp","timestamp.column.name":"dt_update","table.whitelist":"public.dadosdesouripca","topic.prefix":"postgres","validate.non.null":"false","poll.interval.ms":"500","name":"postg-conector-ipc"},"tasks":[{"type":"source"}]}
# {"name":"postg-conector-pre","config":{"connector.class":"io.confluent.connect.jdbc.JdbcSourceConnector","tasks.max":"1","connection.url":"jdbc:postgresql://postgres:5432/postgres","connection.user":"postgres","connection.password":"postgres","mode":"timestamp","timestamp.column.name":"dt_update","table.whitelist":"public.dadosdesouripca","topic.prefix":"postgres","validate.non.null":"false","poll.interval.ms":"500","name":"postg-conector-pre"},"tasks":[{"type":"source"}]}
# C:\Users\gabril\OneDrive\Documents\DOCS\VESTUO5\PO5 XP\ENGENHEIRO DE DADOS\DESAFIO\projeto_desafio_final\confluent curl.exe http://localhost:8083/connectors/postg-conector-ipc/status
# {"name":"postg-conector-ipc","connector":{"state":"RUNNING","worker_id":"connect:8083"},"tasks":[{"id":"0","state":"RUNNING","worker_id":"connect:8083"},"type":"source"]}
# C:\Users\gabril\OneDrive\Documents\DOCS\VESTUO5\PO5 XP\ENGENHEIRO DE DADOS\DESAFIO\projeto_desafio_final\confluent docker exec -it broker bash
# kafka-console-consumer --bootstrap-server localhost:9092 --topic postgres-dadosdesouripca --from-beginning --max-messages 5
# [ppuser@6a73440a910 ~]$ ^C
# [ppuser@6a73440a910 ~]$
# What's next:
# Try Docker Debug for seamless, persistent debugging tools in any container or image + docker debug broker
# Learn more at https://docs.docker.com/go/debug-cli/
# kafka-console-consumer: O termo 'kafka-console-consumer' não é reconhecido como nome de chadlet, função, arquivo de script ou programa operável. Verifique a grafia do nome ou, se um caminho tiver sido incluído, veja se o caminho está correto e tente novamente.
# 31 linha(s) de caracteres
# kafka-console-consumer --bootstrap-server localhost:9092 --topic post ...
# -----
# + CategoryInfo          : ObjectNotFound: (kafka-console-consumer:String) [], CommandNotFoundException
# + FullyQualifiedErrorId : CommandNotFoundException
# C:\Users\gabril\OneDrive\Documents\DOCS\VESTUO5\PO5 XP\ENGENHEIRO DE DADOS\DESAFIO\projeto_desafio_final\confluent docker exec -it broker bash
# kafka-console-consumer --bootstrap-server localhost:9092 --topic postgres-dadosdesouripca --from-beginning --max-messages 5
# [ppuser@6a73440a910 ~]$
```

Figura 15 — Conectores Source (JDBC) registrados e em estado RUNNING no Kafka Connect.

6. Conclusão

O pipeline ETL foi implementado e executado com sucesso de ponta a ponta, atendendo aos três entregáveis solicitados:

- Entregável 1 — as tabelas `dadostesouroipca` e `dadostesouropre` foram carregadas no PostgreSQL, com esquema normalizado e datas convertidas para timestamp (Figuras 1 a 5).
- Entregável 2 — o processamento das camadas Silver e Gold foi feito com Apache Spark, incluindo a consulta Spark SQL de agregação, executado tanto para o IPCA quanto para o PRE-FIXADOS (Figuras 6 a 9).
- Entregável 3 — os dados foram entregues no Amazon S3 organizados em três camadas e particionados: Bronze em `raw-data/` (JSON, `partition=0/`), Silver em `processed-data/` e Gold em `analytics/` (ambas em Parquet) (Figuras 10 a 14).

A integração entre PostgreSQL, Kafka (via Kafka Connect) e Amazon S3, somada ao processamento analítico com Spark SQL, demonstra o ciclo completo de uma solução de Engenharia de Dados no modelo de arquitetura em camadas (medalhão).